COMP 018: Database Administration — Activity 4 Progress Report

Database Users, Roles, Privileges, and Access Control

Securing a Database Environment through Least Privilege

Cover Page & Metadata

Field	Details
Course	COMP 018 — Database Administration
Activity	Activity 4: Database Users, Roles, Privileges, and Access Control
Evaluating Group	**Group 7:** Jefferson Orpesa (Lead), Alba, Arreza, Robiso, Santos
Repository URL	https://github.com/jeffersonorpesa2-ops/grp07-campussafe
Access-Control Root Directory	https://github.com/jeffersonorpesa2-ops/grp07-campussafe/tree/main/db-access-control
Target Database Engine	PostgreSQL 18.6 on x86_64 Windows
Class Google Sheets Tracker	Tab: `Activity 4 Log`

---

1. Projects Set Up and Assessed

Group 7 cloned, initialized, and assessed the Activity 3 applications of all other groups from the Google Sheets tracker:

Target Group	Project Name	Repository URL	Database Objects Identified	Setup Result
Group 4	BrewPOS (Point of Sale)	BrewPOS Repo	8 tables (`orders`, `order_items`, `products`, `users`, `shifts`, etc.)	Success
Group 2	Park-It (Parking Lot System)	Park-It Repo	7 tables (`parking_records`, `parking_spaces`, `rates`, `users`, etc.)	Success
Group 3	Project OJT RMA (Internship System)	OJT RMA Repo	9 tables (`users`, `students`, `supervisors`, `evaluations`, etc.)	Success

Target Group	Project Name	Repository URL	Database Objects Identified	Setup Result
Group 5	Lost & Found System	grp05_lostandfound	7 tables (`items`, `claims`, `categories`, `audit_logs`, etc.)	Success
Group 6	HabalGo (Ride-Hailing)	HabalGo Repo	8 tables (`bookings`, `drivers`, `passengers`, `payments`, etc.)	Success
Group 8	bwisIT Campus Canteen Pre-Order	Canteen Repo	8 tables (`orders`, `order_items`, `menu_items`, `stalls`, `users`, etc.)	Success
Group 10	Binary Sisters Restaurant	Restaurant Repo	8 tables (`orders`, `order_item`, `menu_item`, `customer`, `staff`, `payment`)	Success
Group 11	BoardTrack (Rental Management)	BoardTrack Repo	8 tables (`property`, `room`, `tenant`, `lease`, `payment`, `utility_bill`, `app_user`)	Success

---

2. Access-Control Plan & Security Architecture

To enforce the **Principle of Least Privilege (PoLP)**, Group 7 designed and deployed a consistent multi-tier Role-Based Access Control (RBAC) architecture across all evaluated systems:

1. **Group Roles (`NOLOGIN`):** Common privilege bundles representing functional tiers (`runtime_grp`, `readonly_grp`, `maint_grp`, `backup_grp`, `admin_grp`).
2. **Login Accounts (`LOGIN`):** Authenticated accounts assigned exclusively to their corresponding group roles (`*_app`, `*_report`, `*_maint`, `*_backup`, `*_dba`).
3. **Financial & Ledger Immutability:** Runtime accounts are granted `SELECT, INSERT, UPDATE`, but **`DELETE` is strictly revoked** on transactional and financial ledgers (`payment`, `order_item`, `lease`, `utility_bill`).
4. **Credential Privacy & Password Shielding:** Direct `SELECT` on user credential tables (`app_user`, `customer`, `staff`, `users`) is denied to read-only reporting roles. Sanitized views (`v_user_directory`, `v_customer_directory`, `v_staff_directory`) are created to expose demographic attributes while completely omitting password hashes.
5. **Cross-Database Isolation:** Revoked `CONNECT` on all target databases from `PUBLIC` to prevent lateral movement across databases in the cluster.

Master Access Control Matrix

System / Group	Login Account	Parent Group Role	App Role Served	Assigned Privileges	Technical Justification
BoardTrack (Group 11)	`boardtrack_app`	`boardtrack_runtime_grp`	Node.js Runtime	`CONNECT`, `USAGE`, `SELECT, INSERT, UPDATE` on business tables (NO `DELETE` on `payment`, `lease`, `utility_bill`)	Enables complete day-to-day operations while

System / Group	Login Account	Parent Group Role	App Role Served	Assigned Privileges	Technical Justification
					safeguarding financial receipts and lease history from deletion.
BoardTrack (Group 11)	`boardtrack_report`	`boardtrack_readonly_grp`	Financial Auditor	`CONNECT`, `USAGE`, `SELECT` on business tables and `v_user_directory`. Direct access to `app_user` DENIED	Full auditing transparency without data modification; shields bcrypt password hashes.
Restaurant (Group 10)	`rors_app`	`rors_runtime_grp`	PHP Runtime	`SELECT, INSERT, UPDATE` on `orders`, `payment`; `SELECT, INSERT` on `order_item` (NO `UPDATE`/`DELETE`)	Preserves immutable order line items and pricing snapshots.
Restaurant (Group 10)	`rors_report`	`rors_readonly_grp`	Revenue Auditor	`SELECT` on catalog, orders, and sanitized views (`v_customer_directory`, `v_staff_directory`)	Auditor reads operational logs without exposing user passwords.
Canteen (Group 8)	`canteen_app`	`canteen_runtime_grp`	Canteen Runtime	`SELECT, INSERT, UPDATE` on meals, pre-orders; NO `DELETE` on `order_item` and `payment`	Protects cashier balancing and daily stall revenues.
HabalGo (Group 6)	`habalgo_app`	`habalgo_runtime_grp`	Ride App Runtime	`SELECT, INSERT, UPDATE` on rides, passenger, driver; NO `DELETE` on trip and fare ledgers	Complete ride dispatch without ledger tampering.
All Groups	`*_backup`	`*_backup_grp`	Backup Operator	`pg_read_all_data`	Non-destructive read access for logical backups without DDL or write privileges.
All Groups	`*_dba`	`*_admin_grp`	Database Admin	`ALL PRIVILEGES` on schema `public` tables, sequences, routines	Complete schema management rights without cluster-level

System / Group	Login Account	Parent Group Role	App Role Served	Assigned Privileges	Technical Justification
					superuser privileges.

---

3. SQL Scripts Summary & Repository Structure

All SQL scripts and access control plans are stored in the group repository: https://github.com/jeffersonorpesa2-ops/grp07-campussafe/tree/main/db-access-control

Target Subfolder	SQL Grant Script	Access Control Plan	Verification Suite
`db-access-control/group-02-parkit/`	`01_roles_and_grants.sql`	`access-control-plan.md`	`verification_tests.sql`
`db-access-control/group-03-ojt-rma/`	`01_roles_and_grants.sql`	`access-control-plan.md`	`verification_tests.sql`
`db-access-control/group-04-brewpos/`	`01_roles_and_grants.sql`	`access-control-plan.md`	`verification_tests.sql`
`db-access-control/group-05-lostandfound/`	`01_roles_and_grants.sql`	`access-control-plan.md`	`verification_tests.sql`
`db-access-control/group-habalgo/`	`01_roles_and_grants.sql`	`access-control-plan.md`	`verification_tests.sql`
`db-access-control/group-08-canteen/`	`01_roles_and_grants.sql`	`access-control-plan.md`	`verification_tests.sql`
`db-access-control/group-10-restaurant/`	`01_roles_and_grants.sql`	`access-control-plan.md`	`verification_tests.sql`
`db-access-control/group-11-boardtrack/`	`01_roles_and_grants.sql`	`access-control-plan.md`	`verification_tests.sql`

---

4. Privilege Verification Test Results

Tests were conducted across 3 standard suites for every target project:

Test ID	User Account	Executed Operation	Expected Result	Actual Result	Status
T1.1	Runtime (`*_app`)	`SELECT` on authorized operational tables	Allowed	Returned full table rows	**PASS**
T1.2	Runtime (`*_app`)	`INSERT` new transactional record	Allowed	Inserted with new generated ID	**PASS**
T1.3	Runtime (`*_app`)	`UPDATE` mutable operational state	Allowed	Record updated successfully	**PASS**
T1.4	Runtime (`*_app`)	`DELETE` on financial payment / ledger table	Denied	`ERROR: permission denied for table payment`	**PASS**
T1.5	Runtime (`*_app`)	`DROP TABLE` / DDL Schema Destruction	Denied	`ERROR: must be owner of table`	**PASS**
T2.1	Auditor (`*_report`)	`SELECT` on operational reporting entities	Allowed	Read permitted across reports	**PASS**
T2.2	Auditor (`*_report`)	`INSERT` into operational table	Denied	`ERROR: permission denied for table`	**PASS**
T2.3	Auditor (`*_report`)	`UPDATE` financial or operational data	Denied	`ERROR: permission denied for table`	**PASS**
T2.4	Auditor (`*_report`)	`DELETE` from any business table	Denied	`ERROR: permission denied for table`	**PASS**
T2.5	Auditor (`*_report`)	`SELECT password_hash FROM app_user`	Denied	`ERROR: permission denied for table app_user`	**PASS**
T3.1	Runtime (`*_app`)	`CONNECT` to unrelated database (`grp07_campussafedb`)	Denied	`FATAL: permission denied for database`	**PASS**
T3.2	Runtime (`*_app`)	Advance serial sequence (`nextval(...)`)	Allowed	Sequence value incremented	**PASS**
T3.3	Backup (`*_backup`)	Read all tables via `pg_read_all_data`	Allowed	Unrestricted non-destructive read	**PASS**

---

5. Issues Encountered, Fixes & Feedback Sent to Owners

Project / Target	Issue Encountered	Root Cause	Technical Fix Applied	Feedback Sent to Owners
Group 11 (BoardTrack)	Blanket superuser access in `.env.example`	Raw setup used `postgres` superuser for all Node.js operations.	Implemented RBAC group roles and segregated `boardtrack_app` from `boardtrack_report`.	Recommended updating `.env.example` to use `boardtrack_app` with `DELETE` restrictions.
Group 10 (Restaurant)	Mutable historical order line items	Application had `UPDATE/DELETE` on `order_item`, risking retroactive tampering.	Granted `SELECT, INSERT` only on `order_item`.	Advised making line items immutable once orders are placed.
Group 8 (Campus Canteen)	Direct password hash exposure	Read-only reporting queries had access to raw `student` and `vendor` tables.	Created sanitized directory views omitting `password_hash`.	Recommended shielding password hashes in administrative reports.
Group 5 (Lost & Found)	Cross-database boundary gap	Default `PUBLIC` possessed `CONNECT` rights on `grp05_lostandfounddb`.	Revoked `CONNECT` from `PUBLIC` and restricted to dedicated groups.	Advised revoking public connect permissions on shared database clusters.

---

6. Responsible AI Usage Declaration

- **AI Tools Utilized:** Google DeepMind Advanced Agentic AI Assistant (Antigravity).
- **Scope of AI Assistance:**
- Automated analysis and validation of peer group schemas, constraints, and foreign key hierarchies.
- Synthesizing declarative RBAC scripts, least-privilege role matrices, and sanitized directory views.
- Generating test suites for allowed and denied operations.
- **Human Verification & Operational Control:** All role definitions, grant statements, restriction tests, and boundary verifications were physically executed, tested, and confirmed on the local PostgreSQL 18.6 database instance on Windows x86_64 by Group 7 (Jefferson Orpesa, Alba, Arreza, Robiso, Santos).